

Travaux pratiques : Détection d'objets avec YOLO

Exercice 1 : Comprendre YOLO (Idée générale)

1. Quelle est la différence entre :
 - Une méthode qui analyse **plusieurs régions de l'image** (comme R-CNN)
 - Une méthode qui analyse **l'image en une seule fois** (YOLO)
2. Pourquoi dit-on que YOLO voit l'image de façon **globale** ?
3. Quel est le principal avantage de YOLO par rapport aux autres méthodes ?

Exercice 2 : La grille (Grid)

1. Si une image est découpée en plusieurs cases (grille), que se passe-t-il lorsqu'un objet se trouve dans une case ?
2. Une cellule donne plusieurs informations :
 - Que représente le score de confiance ?
 - Que signifient x, y, largeur et hauteur ?
 - À quoi servent les classes (chat, chien, etc.) ?
3. Si on a plusieurs classes et plusieurs boîtes, que contient la sortie d'une cellule ?

Exercice 3 : Anchor Boxes (Idée simple)

1. Pourquoi utilise-t-on plusieurs formes de boîtes (anchor boxes) ?
2. Comment choisit-on ces formes avant l'entraînement ?
3. Si on augmente le nombre d'anchor boxes, qu'est-ce qui change dans le modèle ?

Exercice 4 : IoU et NMS (Nettoyage des résultats)

1. Que mesure le score IoU ?
2. À quoi sert-il pour vérifier une bonne détection ?
3. Pourquoi YOLO détecte parfois plusieurs fois le même objet ?
4. Comment garder une seule bonne boîte ?

Exercice 5 : Architecture et apprentissage

1. Pourquoi YOLO utilise surtout des couches de convolution ?
2. À quoi sert la fonction de perte (loss function) ?

Exercice 6 : Architecture et Implémentation (TensorFlow/Keras)

Simuler la construction d'un mini-YOLO ou l'utilisation d'un modèle pré-entraîné.

1. Importer les bibliothèques nécessaires (tensorflow, cv2, numpy).
2. Charger un modèle YOLO pré-entraîné (ex : YOLOv8 ou YOLOv5 via Ultralytics ou TensorFlow Hub).

3. Pourquoi utilise-t-on souvent une architecture de type "Darknet" ou des couches de convolution avec "Stride" au lieu de couches "Max Pooling" dans les versions récentes ?
4. Expliquer le rôle de la fonction de perte (Loss Function) de YOLO qui combine :
 - L'erreur de localisation
 - L'erreur de confiance (objet vs pas d'objet)
 - L'erreur de classification

Exercice 7 : Application pratique - Détection sur Image

Appliquer le modèle sur une image réelle.

1. Charger une image de test et la redimensionner à la taille attendue par le modèle (ex : 416×416 ou 640×640).
2. Normaliser les pixels de l'image.
3. Exécuter la prédiction (`model.predict`).
4. Dessiner les boîtes englobantes sur l'image originale en utilisant OpenCV et afficher les noms de classes.

Exercice 8 : Transfer Learning avec YOLO

Adapter YOLO à un nouveau jeu de données.

1. Qu'est-ce qu'un fichier de configuration .yaml ou .cfg dans le contexte de l'entraînement de YOLO ?
2. Pourquoi est-il préférable d'utiliser des poids pré-entraînés (Transfer Learning) plutôt que d'entraîner YOLO à partir de zéro ?
3. Citer trois métriques essentielles pour évaluer un modèle YOLO (indice : mAP, Précision, Rappel).

Exercice 9 : Détection en temps réel (Webcam)

1. Charger la webcam avec OpenCV
2. Appliquer YOLO en temps réel
3. Afficher les FPS (frames per second)

Exercice 10 : Filtrage des objets

Modifier le code pour :

1. détecter uniquement les **personnes**
2. ignorer les autres classes

Sauvegarder :

1. l'image annotée
2. un fichier .txt contenant :
 - classes détectées
 - coordonnées des bounding boxes

Exercice 11 — Détection d'objets sur vidéo avec YOLOv8

1. Chargez un modèle YOLOv8 pré-entraîné (yolov8n.pt) avec YOLO()
2. Ouvrez la vidéo video.mp4 avec cv2.VideoCapture et récupérez ses propriétés : largeur, hauteur et FPS avec cap.get().
3. Créez un VideoWriter en sortie (output.mp4) avec le codec mp4v, les mêmes dimensions et le même FPS que la vidéo source.
4. Dans la boucle de lecture, appelez model.predict(frame) sur chaque frame, puis utilisez results[0].plot() pour obtenir la frame annotée.
5. Écrivez chaque frame annotée dans le fichier de sortie avec out.write(), puis libérez les ressources avec cap.release() et out.release().
6. Affichez dans la console, pour chaque frame traitée, le nombre d'objets détectés (longueur de results[0].boxes).

Exercice 12 : Personnalisation du dataset

7. Créer un dataset personnalisé (ex: fruits)
8. Annoter avec un outil comme LabelImg
9. Entraîner YOLO dessus

Exercice 13 : Comparaison de modèles YOLO

1. Tester :
YOLOv5
YOLOv8
2. Comparer :
 - vitesse
 - précision

Exercice 14 : Détection multi-objets avancée

1. Compter le nombre d'objets détectés par classe
2. Afficher un résumé :

Person: 3
Car: 2
Dog: 1

Exercice 15 : Tracking d'objets

1. Utiliser YOLO + tracking (ex: DeepSORT)
2. Suivre un objet dans plusieurs frames

Exercice 16 : Détection avec seuil dynamique

1. Modifier dynamiquement le seuil de confiance (conf)
2. Observer l'impact sur les résultats

Exercice 17 : Détection + alerte

1. Déclencher une alerte si :
 - une personne est détectée
 - ou un objet spécifique apparaît